TeSpec: Proof-Based Testing of Separation-Logic Specifications for C

Anonymous Authors

***Abstract*—Logical specifications for C describe both functional behavior and ownership of memory. Testing them presents two challenges: quantified properties can remain non-executable after inputs are fixed, and modular verification can abstract away the execution states needed to test an annotation. We present TESPEC, a method for proof-based testing of separation-logic specifications. Typed input bindings select an instance domain. Symbolic execution follows function bodies and loop iterations, while annotations generate obligations on copies of the reached states. Automatic reasoning and proof-assistant completion check those obligations in the original specification logic. Joint heap/value normalization and exact local-storage frames preserve the information observed by subsequent checks. We formalize the instance semantics and mechanize reference trace preservation and nonvacuous acceptance under execution-coverage interfaces. Comparative case studies expose a signed-integer boundary violation missed by the recorded Fulminate configuration, demonstrate proof completion for quantified contracts rejected by runtime instrumentation, and examine automatic versus interactive discharge of spatial obligations. The method makes proof-oriented contracts usable as test oracles while retaining the program states and memory resources that those contracts describe.**

***Index Terms*—Specification testing, symbolic execution, separation logic, proof checking**

I. INTRODUCTION

A specification is useful during development only if programmers can obtain feedback from it. For C, that feedback must cover both functional results and the memory resources that make an execution safe. Separation-logic tools such as CN express these requirements in a single contract [1]. Fulminate makes such contracts executable by translating them into runtime checks [2]. This enables testing before a complete verification proof is available.

Two obstacles remain when testing proof-oriented specifications. First, fixing an input need not make its specification executable. A least-root contract can characterize a result by its relation to *every* nonnegative integer. The program returns a concrete value, but checking that value still requires reasoning about an unbounded domain. Rewriting the contract into a finite monitor is possible for some properties; discovering and justifying that rewrite is additional specification work. Second, the execution used to test an annotation must retain the information produced by the program. A verifier can replace a loop by its invariant or a call by its summary. A test of those annotations needs the actual visited states, including any facts that the annotation omits.

We present TESPEC, a method for *proof-based testing of separation-logic specifications for C*. A test supplies bindings for selected logical inputs. TESPEC executes the program symbolically under those bindings, observes its states at selected specification boundaries, and checks the resulting propositions in the verifier's logic. It executes available function bodies and loop iterations. Invariants and contracts generate obligations on copies of the reached states, while the execution retains the original states. Automatic reasoning handles routine obligations; residual propositions can be completed by an agent or developer and checked by the proof assistant.

This combination makes three kinds of information available to the test oracle: the program's computed values, its owned memory, and the mathematical meaning of the original assertion. A small example exposes why all three matter. Suppose a read function promises to return the value in an owned integer cell, but its implementation returns the negated value. On input `INT_MIN`, the recorded Fulminate configuration exits successfully, whereas TESPEC rejects the arithmetic safety obligation: the positive negation is outside the signed integer range. For quantified least-root and divisibility contracts, the recorded instrumenter cannot infer a finite quantifier bound, whereas proof completion checks the original obligations. Section V reconstructs these cases and their controls.

The central technical problem is preserving the meaning of a test while reusing verification infrastructure. Assertions must observe execution without restricting it. Heap simplification must retain the values later assertions will inspect. An auxiliary assertion may omit unrelated stack slots, but it must not silently discard user-owned memory. TESPEC addresses these problems with copied-state observations, joint heap/value normalization, and exact local-storage frames. A nonempty entry domain and complete execution coverage then connect successful obligation checking to successful instance execution.

Our contributions are:

1) **A stateful proof-based testing method.** Typed input bindings, actual body execution, and copied-state observations turn original separation-logic assertions into test obligations at function, loop, and intermediate boundaries (Section III).
2) **A formal account of instance checking.** We define the tested domain and observation semantics, establish reference trace preservation, and mechanize nonvacuous acceptance under explicit execution-coverage interfaces. Exact frames and joint normalization state which memory and value information a checker must preserve (Section IV).
3) **An implementation and comparative case studies.** The implementation reuses QCP's symbolic execution and Rocq obligations. Evidence-backed cases examine an

integer-boundary miss, quantified properties, and spatial proof completion; a separate evaluation protocol specifies the broader comparisons and their cost units (Sections V and VI).

The testing boundary is explicit: acceptance covers the domain selected by the input bindings. Full verification checks the original contract for all its admissible inputs. We compare these workflows directly, rather than treating a verifier with an added input equality as an execution-preserving test.

II. A Running Example

Consider a routine that replaces an owned integer by its least nonnegative integer square root rounded upward. The contract relates the initial value x, the returned value, and the final cell contents. Its imported Rocq proposition is

$$\mathsf{LeastRoot}(n, r) \equiv 0 \le r \wedge \forall k \in \mathbb{Z}.\, 0 \le k \Rightarrow (r \le k \Leftrightarrow n \le k^2). \tag{1}$$

Listing 1 shows the C body and abbreviated annotations of the archived instance. We use $p \mapsto_{\mathrm{int}} v$ for typed cell ownership; the artifact uses QCP's `data_at` notation.

```
int root_cell(int *p)
/*@ With x
    Require 0 <= x && x <= 100 &&
            data_at(p, int, x)
    Ensure least_root(x, __return) &&
           data_at(p, int, __return) */
{
  int n = *p;
  int r = 0;
  /*@ Inv n == x && 0 <= r && r <= 10 &&
      (r == 0 || (r-1)*(r-1) < n) &&
      data_at(p, int, x) */
  while (r*r < n) { r = r + 1; }
  /*@ Assert least_root(n, r) &&
              data_at(p, int, x) */
  *p = r;
  return r;
}
```

Listing 1. Least-root instance with an intermediate observation. The imported relation is Equation (1).

a) A logical instance.: The binding $\theta = \{x \mapsto 13\}$ fixes the logical input and retains the address p. The spatial precondition describes the cell at that symbolic address. Loop execution produces $r = 0, 1, 2, 3, 4$ at its five invariant boundaries; then the body writes 4 and returns 4. The final observation is

$$\mathsf{LeastRoot}(13, 4) \wedge p \mapsto_{\mathrm{int}} 4.$$

Local scalar storage is handled by the observation frame described in Section III-B. The imported relation remains a proposition in the verification logic. Automatic execution establishes the entry and safety phases, and proof completion checks the remaining functional and observation obligations against their original generated statements.

b) Testing an intermediate specification.: Replacing the assertion by $\mathsf{LeastRoot}(n, r-1)$ changes the observation while leaving the C body and final contract intact. At that boundary the program still has $n = 13, r = 4$, so the candidate asks for $\mathsf{LeastRoot}(13, 3)$. Instantiating its universal quantifier with $k = 3$ exposes the contradiction $13 \le 9$. The failed observation does not replace the reached state or alter the following write. Both variants therefore follow the same program computation.

c) Original assertions and executable reformulations.: The original relation ranges over all nonnegative integers even after n and r are fixed. An executable reformulation is

$$0 \le r \wedge n \le r^2 \wedge \big(r = 0 \vee (r-1)^2 < n\big). \tag{2}$$

The two formulas are equivalent over mathematical integers; the C contract bounds inputs to control machine arithmetic. Constructing and justifying the finite form is one route to a runtime oracle. TESPEC instead retains the original relation as a proposition and reuses its proof library. The checked comparison appears in Section V.

III. Checking Stateful Instances

Figure 1 presents the workflow. The execution path constructs states from the selected inputs and C bodies. The checking path turns those states into immutable obligations, applies automatic reasoning, and submits remaining goals to proof-assistant completion. Three interfaces connect the paths: the entry domain, copied observations, and verdict evidence.

A. Instantiate inputs and execute bodies

Typed bindings specialize the root precondition. A binding can fix a logical integer while leaving an address or part of a data structure symbolic. Entry values remain immutable as C variables and memory change. The entry checker establishes that the resulting domain contains at least one state, using automatic reasoning or an existential Rocq obligation. Execution retains the whole selected domain; an existence witness does not choose the sole test heap.

TESPEC reuses QCP to evaluate C expressions, generate arithmetic and memory-safety obligations, and update the symbolic heap. Known guards select a successor. Unresolved guards retain each successor whose infeasibility has not been established. A call evaluates arguments, allocates fresh parameter slots, and enters the available body. The callee receives fresh logical entry bindings, including in recursive calls. On return, shared writes and the actual return value survive; callee-local slots are retired. A loop repeatedly executes its condition and body, including `break`, `continue`, and early return. Resource exhaustion leaves the execution incomplete.

For example, a helper that increments a cell from 3 to 4 leaves the caller with 4 even if its contract promises only an existential integer greater than 3. The summary is checked against that execution. It does not replace the computed value. This is the distinction from simply strengthening the precondition of a modular verification run with an input equality.

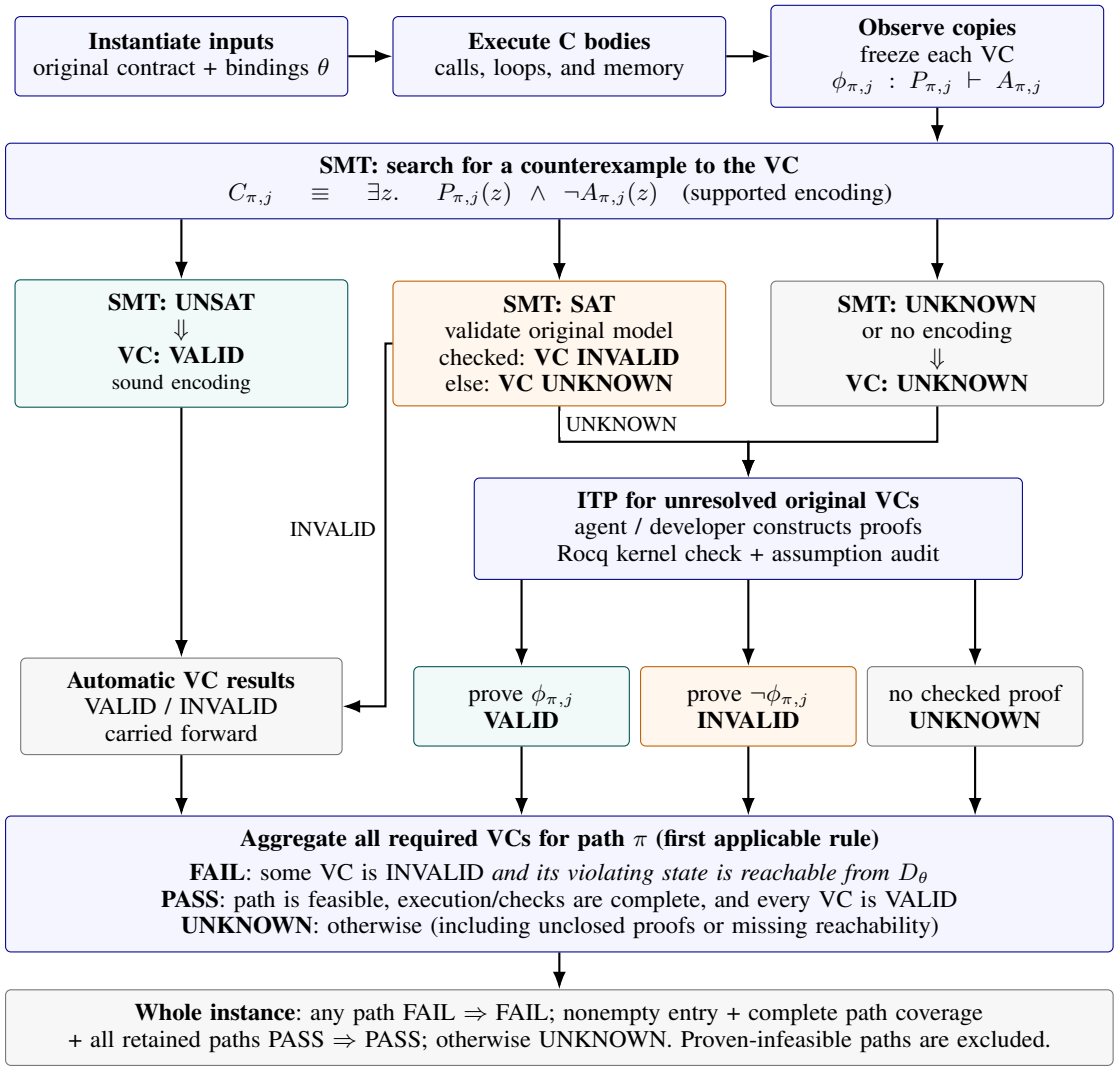


Fig. 1. From symbolic execution to SMT answers, VC status, ITP completion, and path verdicts. SMT checks the *negation* of an entailment, so UNSAT maps to VALID. SAT needs a countermodel in the original interpretation to establish INVALID. ITP closes unresolved original VCs with a checked proof of either polarity. A refuted VC yields path FAIL only with a reachable violating state; one such failure dominates other unresolved VCs. Entry satisfiability has the separate, non-negated meaning shown below the flow.

B. Observe states without changing them

At a selected boundary, let P describe the reached state and A the resolved annotation. The checker emits

$$P \;\vdash\; A * F_{P,A}$$

on a copy and continues execution with P. The frame $F_{P,A}$ contains exactly the eligible local storage omitted by the annotation. Callee contracts and visited loop invariants are observed in this way. A false assertion adds a failing obligation; it does not prune the following execution.

a) Preserve the checked footprint.: The memory model represents scalar C locals as owned cells. An auxiliary annotation can omit unrelated local slots, so the checker resolves its program variables and goal witnesses before selecting the local frame. It preserves the omitted slots' types and current values. If a local pointer holds $\&i$, resolution exposes the i cell before framing and avoids counting it twice. User-owned heap cells are never added merely because the assertion omits them. In particular, a missing returned resource remains observable.

Let m identify eligible local addresses, and split h into h_m and $h_{\neg m}$. For extensional A, the scalar-cell model establishes

$$(A * exact(h_m))(h) \quad \Longleftrightarrow \quad A(h_{\neg m}).$$

For an existential goal, the mask may depend on its witness. A successful witness closes the existential obligation; failure for one chosen witness does not refute all choices.

b) Select observation boundaries.: The controller records which function, loop, and intermediate boundaries the test

checks. Both supported execution policies run available bodies and loops; the richer policy also checks selected internal annotations. Named contract alternatives are resolved by explicit routing, or retained as a disjunction when no alternative is selected. An unresolved boundary match leaves a coverage gap, which prevents complete acceptance.

C. Joint heap and value normalization

Symbolic execution retains values outside its current heap formula: return terms, saved call arguments, entry values, and pending checks. Normalization must preserve their relationship to the heap. Projecting away r from $r = 2 \wedge \mathit{cell}(p, 1)$ can preserve the existentially closed heap assertion while losing the value of a return term that still refers to r. We therefore normalize an assertion P together with its live terms $\bar{t}$. Split the logical environment into immutable root bindings ρ and auxiliary witnesses ξ. Their joint denotation is

$$\mathrm{Obs}_\rho(P, \bar{t})(s, \bar{v}) = \exists \xi.\ P(s, \rho, \xi) \wedge \bar{v} = \mathrm{eval}(\bar{t})_{s,\rho,\xi}.$$

An admitted reduction from $(P, \bar{t})$ to $(P', \bar{t}')$ preserves this relation pointwise for each ρ. Live terms include every value used by the continuation or a selected check; these uses must depend only on the preserved state, root bindings, and live values.

```
Normalize(Gamma, P, liveTerms, budget):
  while budget > 0 and a reduction is admitted:
    resolve the rule in Gamma
    establish its guard from P
    freshen typed witnesses
    replace the complete equivalent expression
    update dependent liveTerms consistently
    simplify exposed computations
    budget := budget - 1
  return (P, liveTerms)
```

Listing 2. Normalize the heap and its live values together.

A guarded rule requires an equation in the selected interpretation,

$$\Gamma \models g(\bar{a}) \Rightarrow \big(H(\bar{a}) \leftrightarrow \exists \bar{w}.\, B(\bar{a}, \bar{w})\big),$$

and validity of $g(\bar{a})$ in the reached state. An unresolved guard retains the original application. Unfolding preserves every pure constraint, resource, alternative, and typed witness. A checked pure consequence may instead be exposed by $H \leftrightarrow H \wedge G$, retaining the spatial predicate.

Fresh witnesses cannot capture live variables. Even a syntactically unused witness of nominal type T carries an existence condition:

$$(\exists w : T.\, P) \leftrightarrow (\mathit{inhabited}(T) \wedge P), \qquad w \notin \mathsf{FV}(P).$$

Integer and list types have canonical witnesses; arbitrary nominal types retain the inhabitance obligation. Shared-base offsets and closed list operations can simplify while addresses remain symbolic. C bounds and arithmetic safety remain required checks.

`DefinitionNormalization.v` proves that composing licensed reductions preserves this joint denotation, nonemptiness, validity, and countermodel queries. Rule licensing is an explicit premise. Budget exhaustion returns the residual expression with its meaning preserved.

D. Discharge immutable obligations

Figure 1 distinguishes three levels: the solver's answer, the validity of an original VC, and the verdict for the execution path. Let $\phi_{\pi,j}$ be the jth obligation on path π:

$$\phi_{\pi,j} \equiv \forall z.\ P_{\pi,j}(z) \Rightarrow A_{\pi,j}(z).$$

Automatic entailment first attempts the original obligation. For SMT, the checker searches for a counterexample by querying $C_{\pi,j} \equiv \exists z.\, P_{\pi,j}(z) \wedge \neg A_{\pi,j}(z)$. The optional `--spatial-smt` mode supplies encodings for supported memory predicates. For a countermodel-preserving encoding, UNSAT establishes VC VALID. A SAT answer establishes VC INVALID once its model is checked in the original memory interpretation. An unconfirmed model, UNKNOWN, or unavailable encoding leaves VC UNKNOWN. Thus satisfiability and validity have opposite polarities here.

Only unresolved VCs require ITP completion. The agent or developer receives the frozen original $\phi_{\pi,j}$ and can construct a proof of $\phi_{\pi,j}$ or of $\neg\phi_{\pi,j}$. A kernel-checked proof under the admitted assumptions establishes VALID or INVALID, respectively. If neither direction is established, the VC remains UNKNOWN; a failed proof attempt supplies no negative evidence. All pending VCs from one test execution form one proof-completion job, with a separate result for each VC. Already decided automatic results are carried forward.

The manifest fixes source hashes, logical dependencies, target interpretation, goal identities, and proof polarity. The optional complete-certificate route replays every original obligation, including entry existence and automatically discharged goals, audits transitive assumptions, and runs an independent kernel check. Automatic evidence and replayed certificates retain distinct labels.

E. Aggregate VC results into path and instance verdicts

A refuted VC does not alone prove that a program reached its countermodel. The path verdict therefore combines VC status with feasibility, execution coverage, and reachability. Let $v_{\pi,j}$ be the final VC status, F_π mean that the path is feasible from the entry domain, K_π mean that its execution and selected checks are complete, and W_π mean that an INVALID VC has an admitted reachable violating state. The path verdict is

$$V(\pi) = \begin{cases} \mathsf{FAIL}, & W_\pi, \\ \mathsf{PASS}, & F_\pi \wedge K_\pi \wedge \forall j.\ v_{\pi,j} = \mathsf{VALID}, \\ \mathsf{UNKNOWN}, & \text{otherwise.} \end{cases}$$

The rules are applied in this order: one confirmed violation suffices for FAIL even if other obligations remain unresolved. A PASS requires all required obligations to be valid. In particular, vacuous VC validity from an infeasible antecedent does not establish a passing execution. Paths proved infeasible are excluded; unresolved feasibility prevents acceptance.

At instance level, any failed path yields FAIL. A nonempty entry domain, complete coverage of its executions, and passing

retained paths yield PASS. Remaining cases yield UNKNOWN. Entry existence has its own query and polarity: SAT means nonempty, UNSAT means an invalid empty instance, and UNKNOWN requires further evidence before acceptance. The CLI retains a separate empty-entry reason when it reports that invalid instance as FAIL. Infrastructure and semantic execution errors keep their own diagnostics. These interfaces connect the workflow to the formal guarantees below.

IV. Instance Semantics and Guarantees

An instance selects an input domain and checks its admitted executions. We formalize this meaning through interfaces for execution, observation, and obligation generation.

A. Instances and selected observations

A concrete configuration σ contains the C environment, heap, control location, and call stack. A logical environment η assigns values to typed logical variables. We write $z = (\sigma, \eta)$; root entry bindings in η remain fixed during execution. The interpretation Γ supplies logical definitions, memory predicates, and the target layout.

Definition 1 (Specification instance)**.** *An instance is $I = \langle c, R, E, \theta, \mathcal{B}, \Gamma, b\rangle$, where c is the body, R and E its entry and return assertions, θ a typed partial binding, $\mathcal{B}$ the selected observation boundaries, and b the execution and proof-search budgets. Its input domain is*

$$D_\theta = \{(\sigma, \eta) \mid \eta \models \theta \wedge (\sigma, \eta) \models_\Gamma R\}. \tag{3}$$

Checking an instance quantifies over all of D_θ. Fixing a stored value can leave its address and other inputs symbolic. An entry witness establishes $D_\theta \neq \varnothing$; it does not replace the domain by that witness. Stronger bindings restrict the domain, but need not preserve nonemptiness.

Heap assertions describe exact owned resources, with separating conjunction interpreted by disjoint composition [3]. At auxiliary boundaries, the resolved check includes the exact local-storage frame of Section III-B. Let $\mathcal{B}_\Gamma(z)$ be the finite list of resolved checks selected at z, including these frames. For a finite trace $\tau = z_0 \cdots z_m$, define

$$\mathsf{Observe}_{\mathcal{B}}(\tau) = \bigoplus_{i=0}^{m} [(z_i, A) \mid A \in \mathcal{B}_\Gamma(z_i)].$$

Concatenation retains order and repeated visits. Resolving a call boundary uses that invocation's logical bindings while preserving the caller's entry values. A checked execution is safe, satisfies every selected observation, and satisfies E on return.

B. Execution and observation interfaces

A symbolic configuration $\langle \ell, P, K, U, \Omega, b\rangle$ contains the control location, represented state assertion, call context, pending observations, accumulated obligations, and remaining budget. Write $[\![P]\!]_\Gamma$ for its represented states and $\to_c$ for the concrete program transition relation.

At a nonterminal program step with successors $P_1, \ldots, P_k$, the executor must establish progress, safety, and coverage:

$$\begin{aligned} &\forall z \in [\![P]\!]_\Gamma.\ \mathsf{Safe}(z) \wedge \exists z'.\ z \to_c z', \\ &\forall z, z'.\ z \in [\![P]\!]_\Gamma \wedge z \to_c z' \Rightarrow z' \in \bigcup_j [\![P_j]\!]_\Gamma. \end{aligned} \tag{4}$$

An unresolved guard retains each successor whose infeasibility has not been established. Coverage may overapproximate executions for acceptance; negative evidence additionally requires realizability.

An observation is an administrative transition:

$$\frac{Q = \mathsf{Resolve}_{\Gamma,K}(A) * F_{P,A}}{\langle P, A :: U, \Omega\rangle \xrightarrow{\text{observe}} \langle P, U, \Omega \cup \{P \vdash Q\}\rangle}. \tag{5}$$

The omitted program context remains unchanged. The continuation retains P, rather than replacing it with Q or requiring the new obligation to hold before execution proceeds.

A finite execution tree contains program-step, observation, normalization, terminal, and unfinished nodes. Terminal nodes establish safety, the return assertion, and absence of further program transitions. Normalization preserves the joint state/-value relation of Section III-C. Reaching an execution bound produces an unfinished leaf. A tree is *boundary-complete* when every selected boundary along each represented execution is paired with an observation node whose antecedent contains the reached state and whose consequent is the correctly resolved check. This connects the selected schedule to the obligation inventory.

C. Reference trace preservation

The reference monitored semantics records observations without consulting their truth.

Theorem 1 (Erasure and lifting)**.** *For any resolved finite boundary schedule $\mathcal{B}$ and state trace τ from z to z', a monitored execution with state trace τ exists if and only if τ is a program execution from z to z'. Its observation list is exactly* $\mathsf{Observe}_{\mathcal{B}}(\tau)$.

Proof. Induct on the finite execution derivation. Each monitored step retains the program transition and appends the selected observations. Erasure removes that list; lifting reconstructs it. No rule requires an observation to hold. □

Changing an observed assertion therefore does not filter program executions in this reference semantics. Erasure, lifting, and the observation inventory are mechanized in `ObservationTrace.v`.

D. Acceptance and negative evidence

Theorem 2 (Nonvacuous instance acceptance)**.** *Suppose a finite, boundary-complete execution tree has root domain $D_\theta \neq \varnothing$, satisfies the execution and normalization interfaces, and has no unfinished leaf. If all generated obligations are valid, every execution from D_θ terminates safely, satisfies E, and satisfies every selected observation.*

TABLE I
EVIDENCE AND INSTANCE RESULTS. EMPTY ENTRIES AND EXECUTION VIOLATIONS RETAIN DISTINCT REASONS.

Layer	Meaning
Entry	SAT: $D_\theta \neq \varnothing$. UNSAT: $D_\theta = \varnothing$. UNKNOWN: nonemptiness unresolved.
VC	VALID: established. INVALID: original VC refuted. UNKNOWN: unresolved.
Test: PASS	Entry SAT, complete execution and boundary coverage, and all required obligations VALID.
Test: FAIL	An empty entry, or a confirmed execution violation; the report retains which reason applies.
Test: UNKNOWN	Neither complete acceptance nor a confirmed failed check.

Proof. Induction on the tree establishes safe termination. Program-step coverage places every concrete successor in a child subtree; normalization and observation nodes preserve the represented continuation. Terminal obligations establish the return assertion. Boundary completeness and valid observation obligations establish each selected check. Entry nonemptiness supplies nonvacuity independently. □

`InstanceChecking.v` mechanizes the finite-tree safety and termination argument; `ObservationTrace.v` establishes satisfaction of a discharged observation inventory. Their connection to C execution and the generated boundary inventory is an implementation obligation.

A confirmed execution violation has existential meaning:

$$\exists z_0 \in D_\theta, z.\ z_0 \to_c^* z \wedge \mathsf{Bad}_{E,\mathcal{B}}(z), \tag{6}$$

where badness is an unsafe state, a violated selected check, or a return state violating E. For a path obligation $P \vdash A$, a positive proof establishes validity for every represented state. A negative proof refutes that obligation. To obtain Equation (6), its violating state must also be realizable from D_θ: an overapproximation may contain unreachable countermodels. A failed proof attempt establishes neither polarity.

Table I summarizes the instance-level results. Figure 1 shows the intervening path aggregation: VC INVALID becomes path FAIL only after reachability is established. An empty entry invalidates the test instance; it is not a counterexample to the postcondition. Entry, obligation, and whole-test results are recorded separately. The emitted initial VC uses a negative-entailment encoding. `PreconditionOracle.v` proves its equivalence to nonemptiness under an explicit classical-logic premise. A concrete entry witness suffices constructively. A Require-only check establishes entry feasibility; an execution check additionally requires the corresponding execution evidence.

V. COMPARATIVE CASE STUDIES

We select cases that expose distinct requirements of a specification test oracle: arithmetic safety, quantified functional relations, and spatial ownership. Each case identifies its source, input, observed tool outcome, and control. These are recorded case studies rather than aggregate results for the full benchmark. The evidence inventory pins the original sources, logs, and proof receipts.

TABLE II
NEGATION MUTANT: CONTROLS AND THE BOUNDARY DISCREPANCY. RUNTIME REJECTION DENOTES FULMINATE EXIT CODE 2.

Initial value	Fulminate	TESPEC
`INT_MIN`	Accept (exit 0)	FAIL (safety)
0	Accept (exit 0)	PASS
$-1, 1, 7$, `INT_MAX`	Runtime rejection	FAIL

A. An integer-boundary violation missed at runtime

The primary `read` program returns the value in an owned cell and preserves that cell. Listing 3 changes only the return expression. The CN and QCP versions express the same ownership and value requirements.

```
int read(int *p)
/*@ With old
    Require data_at(p, int, old)
    Ensure __return == old &&
           data_at(p, int, old) */
{ return -*p; }
```

Listing 3. Negation mutant of read. The contract still requires the original value.

For a 32-bit signed integer, the input $old = -2^{31}$ makes the mathematical negation 2^{31} unrepresentable. C classifies an expression result outside its type's representable range as undefined behavior (Section 6.5, paragraph 5 of the C11 draft [4]). The entry cell is valid; it is the executed negation that violates safety.

The recorded Fulminate pipeline uses `cn instrument --run` with GCC. Both the pipeline and a subsequent invocation of the generated executable exit with code 0; the latter produces no diagnostic. TESPEC reports a satisfiable entry and an invalid safety obligation, yielding FAIL. For this input the range check specializes to the false proposition

$$-2^{31} \leq 2^{31} \leq 2^{31} - 1.$$

The decisive evidence is the arithmetic-safety check, rather than a guessed native return value. The archived log does not record that value.

The controls in Table II distinguish the missed safety violation from ordinary incorrect return values. The correct read implementation also passes at `INT_MIN`. This is a missed violation in the recorded runtime configuration. Compiler and sanitizer configuration form part of that observation; a sanitizer-augmented baseline is a separate configuration in the evaluation protocol. The TESPEC result here is automatic solver evidence, not an independently replayed Rocq certificate.

B. A quantified minimum without a finite monitor

For the `root_cell` example, $n = 13$ produces the value 4, while Equation (1) still quantifies over all nonnegative integers.

The recorded CN instrumenter rejects the original postcondition because it cannot infer an upper bound for the quantified variable. The same limitation appears in the archived Bennet and Darcy configurations. Thus this case exposes an assertion-support boundary, rather than an incorrect runtime answer.

The TESPEC postcondition-only replay initially returns UNKNOWN. Proof completion checks its complete group of 26 original obligations and produces PASS. The proof reasons directly about the quantified relation. As a control, the artifact proves its equivalence to the finite formula in Equation (2); that formula passes Fulminate on the same fixture. The comparison isolates the additional work required to obtain an executable monitor from a valid mathematical specification.

The intermediate assertion provides a second observation site. Replacing $\mathsf{LeastRoot}(n, r)$ by $\mathsf{LeastRoot}(n, r-1)$ preserves the program execution but changes the assertion to $\mathsf{LeastRoot}(13, 3)$. The archived 27-obligation group proves the other obligations and refutes this assertion VC. Its receipt does not admit a complete execution-counterexample certificate, so this is reported as a checked VC refutation. It demonstrates that internal specification changes can be observed even when inputs and outputs are unchanged.

C. Existence and nonexistence in one contract

The divisibility program returns whether `a % b == 0`, under $b > 0$. Its mathematical specification distinguishes two proof tasks:

$$(r = 1 \wedge \exists q \in \mathbb{Z}.\ a = qb) \ \vee \ (r = 0 \wedge \forall q \in \mathbb{Z}.\ a \neq qb).$$

The inputs $(6, 3)$ and $(7, 3)$ exercise both branches. The first proof supplies $q = 2$; the second excludes every integer witness. Both are positive conformance tests. Automatic processing leaves both unresolved, and proof completion checks each complete six-obligation execution group. The receipts record their logical dependencies and the kernel-checking route.

The finite remainder-based CN contract passes both inputs. Adding its logically redundant, unbounded universal consequence causes the tested instrumenter to reject the assertion with a lower-bound inference diagnostic. A separately kernel-checked Rocq lemma establishes the equivalence and the redundancy of that consequence. Together with least-root, this case shows that a formulation change can determine whether a runtime oracle is available, even when the intended property is unchanged.

D. A spatial mismatch resolved by either proof route

Two read variants omit the owned cell from their return contract. The body still retains that cell. In the exact resource interpretation used here, a nonempty owned heap does not satisfy `emp`. An automatically added frame for user heap would hide precisely this specification defect.

Across six integer inputs per variant, the recorded automatic run with spatial SMT disabled leaves 12 cases UNKNOWN. Independent proof-completion jobs refute the original spatial obligations and check entry feasibility. Enabling spatial SMT resolves those same 12 cases as FAIL, with countermodels checked against the finite-heap interpretation and agreement with the existing kernel-checked refutations. The remaining 49 raw classifications are unchanged, including all 22 passing cases. These solver results retain their automatic-evidence label; agreement with a checked refutation does not make the solver output itself a kernel certificate.

Fulminate also detects the missing-ownership variants. This control therefore isolates the effect of TESPEC's proof backend: a residual spatial obligation can represent a real defect, and a stronger solver can decide it without changing execution or rejecting correct controls. The full read group contains 61 variant/input pairs across ten paired configurations. Fulminate matches 60 expected outcomes; TESPEC with per-execution ITP or with spatial SMT matches all 61 under the group's declared policy. Seven of these outcomes are expected memory-access errors for specifications that supply no read permission, and are kept separate from ordinary assertion failures.

VI. EVALUATION DESIGN

The broader evaluation will test whether the case-study mechanisms transfer across programs and predicate families. The protocol fixes program groups, variants, input cases, expected outcomes, and resource accounting before aggregating results. Case studies in Section V are the current evidence; the following comparisons are the study design.

a) Program groups and comparison units.: Each group has a primary program and several implementation or specification variants, with multiple inputs per variant. Existing source variants are retained. Expected behavior is reviewed independently of a tool verdict, including whether a case exercises defined execution, a functional mismatch, a spatial mismatch, or an unsafe operation. Only the primary program requires full verification. Every selected variant/input pair is a test unit. Mutants from one program are correlated observations, so aggregate analysis will report both group-level coverage and per-input outcomes.

b) Comparators and research questions.: Fulminate is the direct runtime-checking baseline on matched CN contracts and inputs. The evaluation will distinguish its recorded configuration from a sanitizer-augmented configuration for arithmetic and memory-safety cases. Original quantified contracts and proved executable reformulations are reported separately. COINS is compared on the shared pure input/output relation under the same model and proof budget. This isolates what execution-derived state and internal observations add. Gillian-C supplies an independent symbolic-testing comparison on a reviewed common feature subset. Unsupported inputs remain visible in coverage figures.

CN and QCP verify each primary program's original contract once, without adding test-input equalities. This comparison asks how early instance feedback and its cost relate to full verification; their denominators are kept separate. Within TESPEC, the four combinations of spatial SMT on/off and ITP on/off measure automatic decision coverage and the contribution of proof completion. The protocol asks: which violations are detected, which original assertions can be

checked, which obligations require proofs, and what each completed decision costs?

c) Outcomes and cost accounting.: Reports retain pass, confirmed violation, unresolved, expected semantic execution error, unsupported input, and infrastructure error. A proof timeout never counts as detection. For proof-producing routes, the report additionally records goal coverage, kernel replay, and admitted assumptions.

Each group receives a result-and-cost table. CN records verification time and peak RSS; Fulminate records instrumentation/compilation and test execution separately. QCP and TESPEC separate VC generation/execution, agent proof filling, proof compilation with assumption audit, and independent kernel checking. One execution's complete residual VC set is one ITP job, including retries. Agent comparisons use `gpt-6.1-sol` with fixed budgets and record input, output, and total tokens. Peak RSS is measured per stage with its process-scope definition; memory peaks are not summed. Repeated isolated runs will report average per-execution cost, retaining raw records and failed attempts. Reused proofs are identified so replay cost is not mistaken for fresh proof-generation cost.

VII. DISCUSSION

a) Choosing a test oracle.: Runtime checking provides immediate feedback when a contract has an executable interpretation. Proof-based testing extends that workflow to original mathematical assertions and reuses the specification's lemmas. Its cost therefore depends on both the execution and the proof route. The spatial read cases show how improving automatic discharge can remove proof work while preserving the execution and its obligations. Quantified contracts illustrate where proof completion remains useful.

b) Validity of the comparison.: The selected cases explain mechanisms and do not estimate their prevalence across projects. Contract pairs must agree on integer widths, ownership, aliasing, and logical definitions. The recorded read discrepancy concerns 32-bit signed negation; no pointer-layout conversion is involved. General C comparisons additionally need matched target layouts and compiler settings. Candidate preconditions also affect which inputs are admitted, so expected behavior and input selection must be fixed independently of those candidates.

c) Scope of the guarantees.: The Rocq development establishes reference observation semantics and the stated execution-tree interfaces. The implementation additionally relies on C-to-VC correspondence, complete successor and boundary generation, and the selected memory interpretation. Automatic results trust the solver encoding; proof certificates check their exported propositions and admitted assumptions. These dependencies are recorded with the evidence rather than inferred from a single success label. Finite instance acceptance covers its selected domain; full verification covers the original precondition.

VIII. RELATED WORK

a) Runtime testing of separation-logic specifications.: CN uses separation-logic refinement types to verify ownership and functional properties of systems C code [1]. Fulminate gives CN specifications a computational interpretation, compiling them into C checks that track ownership transfer during execution [2]. Bennet derives randomized generators for heaps satisfying CN preconditions [5]; Darcy combines randomized choices with SMT constraints in a broader code–specify–test–debug–prove workflow [6]. TESPEC pursues the same early-feedback objective through a proof-based oracle: it symbolically executes a selected input domain and retains assertions as logical obligations. Fulminate is the direct runtime comparator. Bennet and Darcy address input generation and use Fulminate to check the resulting executions, rather than providing independent fixed-input oracles.

b) Testing declarative specifications.: QuickChick integrates property-based testing with Rocq and provides foundations for relating executable testing components to logical specifications [7], [8]. COINS instantiates relational specifications on supplied input/output examples and uses Rocq proofs to establish that those examples are admitted [9]. This is particularly close to TESPEC's treatment of assertions without convenient executable checkers. COINS evaluates a relation on supplied behaviors, while TESPEC executes a C implementation and generates obligations for safety, internal observations, and postconditions. A direct instance proof oracle on the shared pure fragment therefore isolates the benefit of execution-derived observations. QuickChick is complementary when an executable checker is available.

c) Symbolic testing and complete verification.: Gillian provides a memory-model-parametric symbolic execution framework, with C instances supporting symbolic testing and separation-logic verification [10], [11]. It is an independent symbolic-execution comparator for separating the contribution of TESPEC's observation and proof workflow from symbolic execution itself. QCP [12] and CN provide the full-verification comparison. That comparison checks each primary program's original, uninstantiated contract. It measures the cost of establishing the original contract alongside the cost of checking selected instances, without reinterpreting modular verification under a fixed input as a test execution.

d) Semantic foundations and obligation solving.: Exact separation logic distinguishes coverage of program behavior from establishing that a reported behavior is reachable [13]. Verification-preserving inlining studies conditions under which expanding calls and loops preserves verification in automatic separation-logic tools [14]. These results motivate explicit treatment of coverage and realizability. Foundations for translational separation-logic verifiers formalize interfaces between source reasoning, intermediate semantics, and verification backends [15]. TESPEC's reference interfaces similarly separate generation from discharge, while specializing the checked domain and retaining observed states. Finally, separation-logic SMT procedures already decide substantial logical

fragments [16]. Spatial SMT applies this direction to supported finite-heap obligations; proof-assistant completion handles additional reasoning.

IX. Conclusion

TESPEC connects execution-based feedback with proof-oriented C contracts. Input bindings select the tested domain, actual program transitions determine the observed states, and proofs establish their specified properties. Copied observations, joint normalization, and exact local frames preserve this connection through the verification infrastructure. The case studies show why a useful oracle must account for arithmetic safety, original quantified relations, and spatial ownership. The resulting workflow supports testing with the same specifications and logical libraries used for full verification.

Data Availability

The accompanying artifact includes the implementation identities, original case-study sources and logs, proof certificates, and the Rocq reference semantics. The paper's case-study inventory pins evidence by file hash and separates recorded observations from the broader evaluation protocol. The build script regenerates the manuscript and its vector workflow figure.

References

[1] C. Pulte, D. C. Makwana, T. Sewell, K. Memarian, P. Sewell, and N. Krishnaswami, "CN: Verifying systems C code with separation-logic refinement types," *Proceedings of the ACM on Programming Languages*, vol. 7, no. POPL, 2023.

[2] R. Banerjee, K. Memarian, D. Makwana, C. Pulte, N. Krishnaswami, and P. Sewell, "Fulminate: Testing CN separation-logic specifications in C," *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, pp. 1260–1292, 2025.

[3] J. C. Reynolds, "Separation logic: A logic for shared mutable data structures," *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science*, pp. 55–74, 2002.

[4] ISO/IEC JTC1/SC22/WG14, "Programming languages—c," ISO/IEC, Tech. Rep. N1570, 2011, committee draft, April 12. [Online]. Available: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

[5] Z. K. Aamer and B. C. Pierce, "Bennet: Randomized specification testing for heap-manipulating programs," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 3924–3953, 2025.

[6] Z. K. Aamer, R. Banerjee, H. Katsura, D. Kaloper-Meršinjak, D. J. Economou, K. Memarian, D. Makwana, N. Krishnaswami, B. C. Pierce, C. Pulte, and P. Sewell, "Code-specify-test-debug-prove: Flexibly integrating separation logic specification into conventional workflows," *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, 2026.

[7] Z. Paraskevopoulou, C. Hritcu, M. Dénès, L. Lampropoulos, and B. C. Pierce, "Foundational property-based testing," in *Interactive Theorem Proving*, 2015, pp. 325–343.

[8] Z. Paraskevopoulou, A. Eline, and L. Lampropoulos, "Computing correctly with inductive relations," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022. [Online]. Available: https://lemonidas.github.io/pdf/ComputingCorrectly.pdf

[9] F. Yang, X. Li, S. Wang, J. An, Z. Sun, S. Feng, W. Wang, W. Wang, N. Zhan, and F. Xu, "How powerful are LLMs in generating formal program specifications?" in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 145 221–145 249. [Online]. Available: https://proceedings.mlr.press/v306/yang26am.html

[10] J. F. Santos, P. Maksimović, S.-É. Ayoun, and P. Gardner, "Gillian, part I: A multi-language platform for symbolic execution," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI. ACM, 2020, pp. 927–942. [Online]. Available: https://j3fsantos.github.io/papers/gillian.pdf

[11] P. Maksimović, S.-É. Ayoun, J. F. Santos, and P. Gardner, "Gillian, part II: Real-world verification for JavaScript and C," in *Computer Aided Verification*, ser. CAV. Springer, 2021. [Online]. Available: https://vtss.doc.ic.ac.uk/publications/Maksimovic2021Gillian.html

[12] X. Wu, Y. Feng, X. Lu, T. Lin, K. Liu, Z. Wang, S. Wu, L. Xie, C. Yang, H. Zhong, Z. Zhang, J. Li, N. Zhan, Z. Hu, and Q. Cao, "QCP: A practical separation logic-based C program verification tool," *arXiv preprint arXiv:2505.12878*, 2026. [Online]. Available: https://arxiv.org/abs/2505.12878

[13] P. Maksimović, C. Cronjäger, A. Lööw, J. Sutherland, and P. Gardner, "Exact separation logic: Towards bridging the gap between verification and bug-finding," in *37th European Conference on Object-Oriented Programming*, ser. Leibniz International Proceedings in Informatics, vol. 263. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, pp. 19:1–19:27. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2023.19

[14] T. Dardinier, G. Parthasarathy, and P. Müller, "Verification-preserving inlining in automatic separation logic verifiers," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, 2023. [Online]. Available: https://gauravpartha.ch/papers/inlining_oopsla23.pdf

[15] T. Dardinier, M. Sammler, G. Parthasarathy, A. J. Summers, and P. Müller, "Formal foundations for translational separation logic verifiers," *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, 2025. [Online]. Available: https://gauravpartha.ch/papers/translational_foundations_popl25.pdf

[16] A. Reynolds, R. Iosif, C. Serban, and T. King, "A decision procedure for separation logic in SMT," in *Automated Technology for Verification and Analysis*, ser. ATVA. Springer, 2016, pp. 244–261. [Online]. Available: https://cvc4.github.io/publications/2016/RIS%2B16.pdf